

End-to-End LSTM Neural Model for Java Code Summarization

Notebook Write-Up

Sam Bennett

March 25, 2026

1 Overview

This write-up summarizes the three major components implemented in the notebook `GenAISWE_Assignment_2_Sam`. The notebook presents a full end-to-end pipeline for Java code summarization using an LSTM encoder-decoder model. The workflow is divided into 4 parts:

1. mining software repositories to construct training and validation data
2. embedding training and validation data using CodeT5
3. training and validating an LSTM summarization model
4. evaluating the trained model on a held-out test set with multiple generation metrics

The overall goal of the pipeline is to learn to generate concise natural-language summaries for Java methods by combining mined repository data, pretrained CodeT5 token embeddings, and a sequence-to-sequence LSTM architecture.

2 Java Code Summarization Dataset Building

First, I constructed the dataset used for model development. This stage focuses on automatically mining Java repositories, extracting method-summary pairs, normalizing and filtering the resulting examples, and producing train/validation splits suitable for downstream neural modeling.

2.1 Repository Discovery and Metadata Screening

Candidate repositories are retrieved from GitHub using the search API with a Java-language and minimum star-count constraint of 100. I use repository descriptions and, when needed, README text to determine whether a repository appears to be documented in English, and thus is probably developed by english speaking developers. This filtering step is important because the final task is English code summarization, so non-English metadata is likely to introduce noise into both the mined summaries and the resulting model. Only repositories that pass this language screen are kept for cloning. At the end of these phase, I have collected a curated set of 700 Java repositories

URIs, that reference code that is both popular (possibly denoting a high-quality nature) and more likely to contain usable English documentation.

2.2 Repository Cloning and Java File Selection

After repository selection, each repository is shallow-cloned locally using `git clone -depth 1`. The shallow clone strategy reduces storage cost and speeds up collection while still providing access to the latest source files needed for dataset construction. The notebook then walks each cloned repository and selects 40 Java files per repo while excluding directories and paths likely to contain poor-quality training examples, such as:

- test code,
- example/demo code,
- generated files, and
- other non-production artifacts.

If a repository contains more than 40 Java files, a random sample is taken so that large repositories do not dominate the mined corpus.

2.3 Method and Summary Extraction

From the selected Java files, I parse source code using `javalang` and extract individual method bodies. For each method, my code attempts to recover an associated top-level comment or docstring-like description that can serve as a target summary. This is done through regex parsing that looks for both java docstrings above method signatures and top-level comments directly below method signatures. At this stage the pipeline produces aligned pairs of the form:

(Java method source, natural-language summary)

The extraction process also keeps contextual information needed for later auditing, including repository name, file path, method name, and method location. This is useful for diagnosing why some candidates were accepted or rejected to tune sample pair filtering in the next phase.

2.4 Normalization, Filtering, and Deduplication

Once raw pairs have been extracted, I apply several quality-control steps:

- code is normalized into single-line text representations
- summaries are normalized and lowercased

- malformed or incomplete methods (not containing both closing and open bracket around method body) are removed. *Note this naturally leads to the exception of interface signatures and their top level comments, but since these blocks of code are relatively simple, I decided to exclude them in my training and validation data.*
- noisy summaries that look like code fragments are filtered out
- non-English summaries are discarded
- duplicate method–summary pairs are removed

These filtering operations are critical because mined repository data is naturally noisy. Without these checks, the model would train on mismatched, duplicated, or low-quality examples that would likely hurt both convergence and final generation quality.

2.5 Audit Files and Data Collection Reporting

To support transparency in the mining pipeline, I write audit files for dropped or questionable samples. These artifacts make it possible to inspect why examples were excluded, such as missing comments or filtering failures. These audit files were vital to tuning the filtering logic to ensure the proper targets were met.

2.6 Handling Data Shortfalls with CodeXGLUE Backfilling

If the mined repositories do not yield enough usable examples to meet the target dataset size 51k sample pairs, I backfill the shortfall using the CodeXGLUE `code_to_text` dataset. For these fallback examples, my code collects the summary for each method included in CodexGLUDE code-to-text from either the `docstring` or `tokenized docstring content` dataset fields. This strategy ensures that the pipeline can still reach the desired training and validation set sizes even when repository mining alone is insufficient. The backfilled examples are normalized and filtered (including deduplication) using logic consistent with the main mining pipeline so that the final dataset remains as coherent as possible.

2.7 Train/Validation Split and Saved Outputs

After filtering and deduplication, the examples are shuffled and split into training and validation sets. I save the final parallel text files as:

- `train_code.txt`
- `train_summary.txt`
- `val_code.txt`
- `val_summary.txt`

Each file preserves alignment between source code and summary examples.

3 Embedding Train and Validation Data with CodeT5

Before LSTM training, the notebook uses a pretrained CodeT5 embedding pipeline to tokenize the code and summary corpora and map tokens to dense vectors. Code and summaries are embedded separately, and the outputs are stored in serialized `.pt` files for both the training and validation splits. These files contain token ids, tokenizer metadata, and the embedding matrix used later by the LSTM model. This design allows the LSTM system to leverage a pretrained token space rather than learning token embeddings from scratch, which should improve semantic grounding and reduce the burden on the downstream model.

4 Train and Validate LSTM

It is now time to train a sequence-to-sequence LSTM model on the prepared dataset and saved the best snapshot. Here I cover data loading, model construction, optimization, validation, and check-pointing.

4.1 Training Setup and Constants

First I define constants controlling important training behavior, including:

- batch size
- maximum sequence length
- hidden dimension
- number of epochs
- evaluation frequency
- execution device (CPU or CUDA)

I built a smoke-test mode into the notebook to verify the training pipeline quickly on a very small subset of the dataset. Unfortunately, I struggled to port my notebook to a properly configured remote environment with access to a GPU, so I was not able to collect results on a full-size experiment in time. I will make sure to develop in a local environment consistent to a accessible GPU enabled environment in the future to ensure proper collection of results.

4.2 Dataset Loading and Batch Construction

The embedded train and validation `.pt` files are loaded into a custom `CodeSummaryDataset`. Each example contains aligned token-id sequences for code and its corresponding summary. I check that the code and summary files are compatible by validating that they share the same tokenizer name, vocabulary size, embedding dimension, and special token ids. A custom collate function pads

variable-length source and target sequences to produce rectangular tensors for batched training. This is necessary because Java methods and summaries naturally vary in length.

4.3 Tokenizer Recovery for Decoding and Validation

The tokenizer used to produce the embeddings is reloaded from the saved metadata. This same tokenizer is then used to decode generated token sequences back into text during validation. Reusing the original tokenizer ensures that token ids are interpreted consistently across preprocessing, training, and evaluation.

4.4 LSTM Encoder–Decoder Architecture

The model I implemented in the notebook is a standard encoder–decoder LSTM adapted to reuse pretrained CodeT5 embeddings. Its main components are:

- **Embedding layer:** initialized directly from the pretrained CodeT5 embedding matrix.
- **Projection layer:** maps embedding vectors to the LSTM hidden size when the dimensions differ.
- **Encoder LSTM:** reads the input code token sequence and compresses it into final hidden and cell states.
- **Decoder LSTM:** generates the summary autoregressively from the encoded representation.
- **Output layer:** projects decoder hidden states to vocabulary logits for next-token prediction.

The model stores the padding, start-of-sequence, and end-of-sequence token ids so that generation and loss computation remain consistent with the tokenizer metadata.

4.5 Training Procedure

Training follows a teacher-forcing setup. During each step:

1. the source code token sequence is embedded and encoded
2. the decoder is fed the gt summary prefix
3. the model predicts the next token at each position
4. the loss is computed against the gt target summary

The notebook trains for multiple epochs, running loss is accumulated throughout training, and evaluation is performed at fixed step intervals consistent with lab LSTM implementation.

4.6 Validation with BLEU-1

Every 500 steps, the model switches to evaluation mode and generates summaries without gradient computation on the entire validation set. Each generated summary is decoded into text and compared against the reference summary using BLEU-1. Using BLEU-1 during validation gives a lightweight measure of current model quality that is cheap enough to compute regularly while still providing feedback on whether the model is learning to produce relevant summary tokens.

4.7 Checkpointing and Early Stopping

The notebook tracks the best observed validation BLEU-1 score and saves the corresponding checkpoint to a stable file in the checkpoint directory. If validation performance fails to improve for 3 validation rounds, the training loop stops. This helps prevent unnecessary training once progress has plateaued and preserves the best-performing model state for final testing.

5 Evaluate Model on Given Test Set and Compute Metrics

Finally, I reconstruct the trained model from the checkpoint, generate summaries for the held-out test set, and computes a rich collection of evaluation metrics.

5.1 Model and Metadata Reconstruction

To guarantee consistency with training, the notebook reloads the saved embedding metadata and tokenizer information from the training artifacts. It then reconstructs the LSTM model with the same embedding matrix, special token ids, and architecture settings before loading the best saved checkpoint weights. The tokenized test set is read from a CSV file that includes raw code, reference summaries, and serialized token-id lists for both code and summary text. These ids are parsed back into lists so the evaluation pipeline can batch the examples and decode both predictions and references consistently with the training and validation data.

5.2 Test-Time Summary Generation

The notebook defines a `TestCodeDataset` and pads batches of source code token sequences for inference. It then generates one summary per test example using the trained model and decodes the outputs back into plain text. For each example in test set, the evaluation pipeline stores:

- the original code snippet
- the gt reference summary
- the generated summary
- example identifiers or supporting metadata.

These generated outputs are saved to `lstm_test_predictions.json`, which provides a persistent record of model behavior on the held-out test set.

5.3 Evaluation Metrics

Unlike the validation stage, final testing uses several complementary metrics:

- **BLEU-1 through BLEU-4:** measure n-gram overlap between predictions and references.
- **METEOR:** captures unigram matching with some flexibility beyond raw n-gram precision.
- **BERTScore:** measures semantic similarity using contextual embeddings and reports precision, recall, and F1.
- **SIDE:** estimates how well a generated summary semantically aligns with the source code itself, rather than only the reference summary. To facilitate the use of side, I download the hard-negatives embedding model snapshot from <https://github.com/antonio-mastropaolo/code-summarization-metric>.

Together, these metrics provide a broader view of model quality. BLEU emphasizes lexical overlap, METEOR adds a more forgiving lexical comparison, BERTScore evaluates semantic similarity, and SIDE attempts to assess whether the generated summary remains faithful to the code being summarized.